IMPLEMENTATION HANDOFF

LLM Token-Balancing Gateway

OpenAI-compatible API contract, routing controls, full request lifecycle, architecture, persistence, errors, tests, milestones, and acceptance criteria.

VERSION 0.1 | 9 AUGUST 2026
Status: implementation-ready specification

Executive implementation brief

Build a single-process async Python gateway that presents an OpenAI-compatible surface while selecting, invoking, validating, repairing, and escalating across heterogeneous providers. SQLite is the v0.1 default; PostgreSQL must work without domain redesign. Machine-learned routing, a web UI, distributed workers, and multi-tenant billing are out of scope.

Normative language. MUST, MUST NOT, SHOULD, and MAY are requirements terms.

A request succeeds only when a generation attempt passes its required validation gates. Provider success alone is not gateway success.

Non-negotiable invariants

- Privacy, provider policy, capabilities, context fit, and risk quality floors are hard gates; price never overrides them.
- Reserve estimated monetary cost atomically before every billable invocation; settle or release on every exit.
- Never exceed the caller's request cost ceiling. Fail before invoking another model when no feasible route remains.
- Freeze policy version and route plan. Persist immutable attempt and validation telemetry.
- Do not store raw prompts or outputs by default; store keyed hashes and redacted operational metadata.

v0.1 decisions

Decision	Contract
Generation APIs	Implement both POST /v1/chat/completions and POST /v1/responses.
Inspection	POST /route/inspect; no provider call or reservation.
Controls	Top-level gateway object preferred; X-LLM-* headers supported and override body.
Selectors	auto aliases plus explicit gateway model IDs; explicit IDs still pass governance.
Streaming	Route fixed before first event; no transparent escalation after visible output.
Persistence	SQLAlchemy 2.x, Alembic, SQLite WAL, PostgreSQL compatibility.
Provider layer	ProviderAdapter abstraction; LiteLLM may normalize HTTP but is not the router.

1. Public API contract

Base conventions

Concern	Contract
Authentication	Authorization: Bearer . Invalid/missing keys: 401.
Content type	application/json UTF-8; unsupported type: 415.
Request IDs	Accept valid X-Request-ID or create req_; always return it.
Idempotency	Idempotency-Key on non-stream POSTs. Same canonical input replays; different input: 409.
Timeout	Use lower of client max_latency_ms and deployment maximum.
Compatibility	Strict gateway fields; safely ignorable standard fields may be ignored with telemetry.
Observability	X-LLM-Request-ID always; route/cost summary only when authorized.

Endpoints

Method/path	Purpose	v0.1
POST /v1/chat/completions	Chat Completions-compatible generation	Required
POST /v1/responses	Responses-compatible generation	Required
GET /v1/models	Selectors and enabled explicit model IDs	Required
POST /route/inspect	Classify, filter, rank, explain	Required
GET /health/live	Process liveness	Required
GET /health/ready	DB, registry, required adapter readiness	Required
GET /v1/requests/{id}	Sanitized status; admin scope	Recommended

Model selectors

Selector	Meaning
auto	Balanced objective using effective controls.
auto-cheap	Minimize expected total effective cost subject to quality floor.
auto-fast	Minimize predicted latency subject to quality and cost.
auto-quality	High quality target unless critical is explicit.
auto-private	Force privacy=confidential.
	Manual override; fallback only when allow_fallback=true.

2. Gateway controls

Clients SHOULD send a top-level `gateway` object. Equivalent headers support clients unable to extend bodies. Headers override body; security/deployment policy overrides all caller preferences.

Body/header	Type/default	Meaning
quality / X-LLM-Quality	economy|standard|high|critical; standard	Minimum quality floor and validation level.
privacy / X-LLM-Privacy	public|confidential; confidential	Confidential excludes public-data handling tiers.
max_cost / X-LLM-Max-Cost	decimal USD; deployment default	Hard total request monetary ceiling.
max_latency_ms / X-LLM-Max-Latency	integer; deployment default	End-to-end deadline.
provider_allow / provider_deny	string[]	Allow intersection; deny wins.
required_capabilities	string[]; inferred	tools, json_schema, vision, streaming, reasoning.
allow_fallback	bool; true auto, false explicit	Allows another eligible model.
max_attempts	1..5; policy <=3	Generation and repair cap; validator calls excluded but cost included.
validation	auto|none|deterministic|independent	none only low risk; never bypass hard schema gates.
task_class / risk	hint; inferred	Caller may raise risk, never lower inferred risk.
dry_run	bool; false	Generation endpoint behaves as inspection.
metadata / debug	object / bool	Size-limited tags; debug requires scope.

Precedence

```
deployment security policy
 > authenticated client policy
 > X-LLM-* headers
 > gateway body
 > model-alias defaults
 > system defaults
```

- Reject invalid enums, NaN/negative cost, max_attempts outside 1..5, oversized metadata, and contradictory capabilities.
- Parse/enforce money using Decimal, never binary floating point.
- For privacy, risk, and quality, effective value is the strictest applicable value.

3. Chat Completions and Responses

POST /v1/chat/completions

```
{
  "model":"auto",
  "messages":[{"role":"user","content":"Return JSON with a title."}],
  "max_tokens":300, "stream":false,
  "response_format":{"type":"json_object"},
  "tools":[],
  "gateway":{"quality":"standard","privacy":"confidential",
             "max_cost":"0.050000000","max_latency_ms":12000}
}
```

Support common fields: model, messages, temperature, top_p, max_tokens/max_completion_tokens (mutually exclusive), stop, n, stream/options, penalties, seed, response_format, tools/tool_choice, parallel_tool_calls, user, metadata. v0.1 rejects n != 1; audio generation is out of scope.

```
{
 "id":"chatcmpl_<uuid>","object":"chat.completion","model":"auto",
 "choices":[{"index":0,"message":{"role":"assistant","content":"..."},"finish_reason":"stop"}],
 "usage":{"prompt_tokens":81,"completion_tokens":42,"total_tokens":123},
 "gateway":{"request_id":"req_<uuid>","resolved_model":"provider/model",
            "attempts":2,"validation":"PASS",
            "cost":{"currency":"USD","actual":"0.003120000"},"latency_ms":1840}
}
```

POST /v1/responses

```
{
 "model":"auto-quality","instructions":"Be concise.",
 "input":[{"role":"user","content":[{"type":"input_text","text":"Review this SQL."}]}],
 "max_output_tokens":1200,"stream":false,"tools":[],
 "gateway":{"privacy":"confidential","risk":"high","max_cost":"0.20"}
}
```

Support model, instructions, input, max_output_tokens, sampling, stream, tools/tool_choice, text.format, reasoning effort when supported, metadata, and gateway. Reject background mode, remote MCP/computer-use execution, and provider-hosted files unless separately enabled.

Both endpoints normalize to one immutable CanonicalRequest. Endpoint serializers are pure adapters; routing, budgets, validation, and escalation never operate on framework/provider objects.

4. Streaming and route inspection

Streaming invariants

- Finish classification, planning, eligibility, and reservation before committing response headers.
- After any provider output becomes client-visible, MUST NOT switch models.
- Failure before first visible token may retry/fallback within deadline, attempts, and cost.
- Failure after first token ends stream and persists FAILED_PARTIAL.
- High-risk/full-output validation must buffer or reject streaming with validation_requires_buffering.
- Disconnect cancels adapter, settles known usage, releases unused reservation.

POST /route/inspect

Accept a Chat- or Responses-shaped payload plus gateway controls. Run authentication, normalization, classification, snapshotting, filtering, scoring, and validation planning. Do not reserve budget, call a generation/validator provider, or create attempt rows.

```
{
 "object":"gateway.route_inspection","request_id":"inspect_<uuid>",
 "classification":{"task_class":"sql_review","complexity":4,"risk":"high",
                   "privacy":"confidential","confidence":0.91},
 "effective_controls":{"quality":"critical","max_cost":"0.200000000"},
 "policy":{"id":"sql-high-risk-v3","version":3},
 "candidates":[
   {"rank":1,"model":"included/code-strong","eligible":true,
    "estimated_cost":"0","effective_cost":"0.0041",
    "predicted_latency_ms":2900,"predicted_pass_probability":0.86}
 ],
 "excluded":[{"model":"free/flash","reasons":["privacy_mismatch","below_quality_floor"]}],
 "planned_validation":["sql_parser","sql_safety","independent_review"],
 "estimated_route_upper_bound":"0.118","warnings":[]
}
```

- Disclosure of scores/provider IDs requires debug scope.
- Inspection is advisory because health, quota, price, and budget can change.
- Same canonical input and snapshots must rank deterministically; tie-break by score, predicted cost, model ID.

5. Canonical domain and architecture

```
CanonicalRequest
 request_id; endpoint; requested_model; conversation; system_instructions
 tools; tool_choice; output_format; sampling; max_output_tokens; stream
 controls; input_hash; estimated_input_tokens; client_id; idempotency_key

RequestFeatures
 task_class; complexity 1..5; risk; privacy; verifiability; freshness
 required_capabilities; expected_output_tokens; classifier/version/confidence

RoutePlan
 policy_id/version; registry_snapshot_at; candidates; validation_plan
 max_generation_attempts; max_same_model_repairs; estimated upper bound
 deadline_at; rationale_codes
```

Core interfaces

```
Router.plan(request, features, snapshot) -> RoutePlan
Router.next_step(plan, history, validation) -> RouteDecision
PolicyEngine.evaluate(request, features, models, budgets, quotas) -> Eligibility
ModelRegistry.snapshot() -> RegistrySnapshot
BudgetManager.reserve/settle/release(...)
ProviderAdapter.generate/stream(...); capabilities()
Validator.validate(request, candidate, context) -> ValidationResult
```

Flow

```
Client -> API/auth/normalizer -> classifier -> policy + registry/budget/quota/health
       -> route planner -> orchestrator state machine
          -> reserve -> provider -> settle -> validate
          -> success | repair | fallback | escalation
       -> endpoint serializer -> client

Cross-cutting: repositories | audit/metrics/tracing | redaction
```

Suggested package layout

```
src/gateway/
 api/{app,auth,errors,chat,responses,inspect,health}.py
 domain/{requests,models,routing,validation,budgets,errors}.py
 services/{normalizer,classifier,policy,router,orchestrator}.py
 providers/{base,litellm_adapter,commandcode_cli}.py
 validators/{base,json_schema,code,sql,llm_judge}.py
 persistence/{models,repositories,unit_of_work,migrations}.py
 telemetry/{logging,metrics,tracing,redaction}.py
tests/{unit,integration,contract,e2e,fixtures}/
```

6. Persistence and budgets

Retain the defined models, model_prices, routing_policies, requests, attempts, validations, budgets, and quota_snapshots tables. Add operational reservation and idempotency tables.

Table	Required details
requests	endpoint, requested_model, normalized_controls_json, state, terminal_reason, deadline_at, policy_version, registry_snapshot_at.
attempts	reservation_id, route_rank, emitted_output, attempt_kind, retry_of_attempt_id; unique request sequence.
validations	required flag and aggregate effect; LLM judge links through validator_attempt_id.
budget_reservations	request/attempt/budget, reserved/settled amounts, status, expiry; unique attempt+budget.
idempotency_records	client, key, input_hash, request, state, response_ref, expiry; unique client+key.
quota_snapshots	Append-only latest per provider/model/window; staleness may exclude routes.

Atomic budget algorithm

```
estimate = price(input_estimate, expected_or_max_output) + request_fee
assert request_cost_so_far + estimate <= effective_max_cost
BEGIN ATOMIC
 lock applicable budgets in stable scope order
 assert each hard_limit - spent - reserved >= estimate
 increment reserved; insert active reservation rows
COMMIT
invoke provider (never hold DB lock)
BEGIN ATOMIC
 active -> settled/released
 decrement reserved; increment spent by actual monetary cost
COMMIT
```

- PostgreSQL uses row locks; SQLite uses BEGIN IMMEDIATE and a short busy timeout.
- Under-estimated actual cost is settled accurately and emits a high-severity metric; further attempts stop if ceiling is exhausted.
- A reconciler expires orphan reservations and marks stale invocations abandoned without blindly repeating ambiguous billable calls.

Retention

Default: telemetry 90 days; idempotency response reference 24 hours; raw content off. Hash inputs/outputs with a deployment-specific keyed digest. Errors and metadata are redacted immediately.

7. Full request lifecycle/state machine

```
RECEIVED
 -> AUTHENTICATING -> REJECTED
 -> NORMALIZING    -> REJECTED
 -> CLASSIFYING    -> FAILED
 -> PLANNING       -> REJECTED_NO_ROUTE
 -> READY -> RESERVING -> REJECTED_BUDGET
 -> INVOKING -> RETRY_WAIT | VALIDATING | FAILED | FAILED_PARTIAL
 -> VALIDATING -> SUCCEEDED | REPAIR_PLANNING | ESCALATION_PLANNING
 -> REPAIR_PLANNING / ESCALATION_PLANNING -> RESERVING | FAILED_EXHAUSTED

Any non-terminal -> CANCELLED when safely observed.
Terminal: SUCCEEDED, REJECTED, REJECTED_NO_ROUTE, REJECTED_BUDGET,
FAILED, FAILED_PARTIAL, FAILED_EXHAUSTED, CANCELLED, EXPIRED.
```

State	Entry action	Exit
RECEIVED	Assign ID/deadline; claim idempotency.	Proceed or replay.
AUTHENTICATING	Validate key/scopes/client policy.	Authenticated or 401/403.
NORMALIZING	Parse endpoint/controls; infer capabilities; estimate/hash.	Canonical request or 400/413/415.
CLASSIFYING	Task/risk/privacy/complexity vector.	Frozen features or safe configured fallback.
PLANNING	Snapshot policy/registry/quota/budgets; filter/rank.	Plan or no-route.
RESERVING	Atomically reserve next estimate.	Active reservation or budget denial.
INVOKING	Create attempt; adapter call under remaining deadline.	Result/retry/cancel/partial.
VALIDATING	Ordered required validators; persist each.	Pass, repair, escalate, exhaust.
REPAIR_PLANNING	Original request + concise failures; candidate only if useful.	One repair or escalate.
ESCALATION_PLANNING	Next candidate; re-check live constraints.	Reserve or exhaust.

8. Transition details and stop rules

Provider outcomes

Condition	Action
Complete success	Settle; persist usage/output hash; validate.
429/retryable 5xx before output	Settle/release; bounded jitter; then same-tier fallback.
Timeout before output	Cancel; settle/release; fallback only if time remains.
Auth/config error	Open circuit; no blind retry; alternate route or fail.
Context/capability rejection	FAIL_CAPABILITY; compatible fallback; health signal.
Partial visible stream	Settle usage; terminal FAILED_PARTIAL; no escalation.

Validation aggregation

```
FAIL_GROUNDING > FAIL_CAPABILITY > FAIL_QUALITY > FAIL_REPAIRABLE > INDETERMINATE > PASS
```

Result	Default next action
PASS	Complete.
FAIL_REPAIRABLE	Repair same model once if useful and allowed.
FAIL_QUALITY	Escalate using original input plus failures.
FAIL_CAPABILITY	Compatible fallback; no repair.
FAIL_GROUNDING	Grounding route if configured; else escalate/fail.
INDETERMINATE	Independent validation when risk requires; else policy.

Stop before every invocation

- Attempt or same-model repair cap reached; no untried eligible candidates.
- Remaining deadline cannot cover predicted attempt plus validation margin.
- Next estimated total exceeds request max_cost or scoped hard budget.
- Eligibility changed because of policy, registry, health, privacy, capability, or quota.
- Client cancelled; circuit open; idempotency owner completed elsewhere.

9. Error handling

Use an OpenAI-style envelope for generation and inspect. Never expose secrets, raw provider payloads, internal paths, SQL, or prompt text.

```
{
 "error":{"message":"No eligible route satisfies privacy and cost constraints.",
          "type":"gateway_policy_error","param":"gateway.max_cost",
          "code":"no_eligible_route"},
 "gateway":{"request_id":"req_<uuid>","retryable":false}
}
```

HTTP	Codes	When
400	invalid_request, invalid_gateway_control, validation_requires_buffering	Malformed/conflicting or unsupported semantics.
401	invalid_api_key	Authentication failure.
403	policy_denied, model_not_allowed	Authenticated but disallowed.
404	model_not_found, request_not_found	Unknown explicit resource.
409	idempotency_conflict, request_in_progress	Key conflict/race.
413	context_too_large, request_too_large	Payload/context cannot fit.
422	no_eligible_route	Valid but constraints unsatisfiable.
429	budget_exceeded, quota_exhausted, rate_limited	Capacity constraint; Retry-After when known.
500	internal_error, persistence_error	Unexpected gateway fault.
502	provider_error, invalid_provider_response	Upstream exhaustion.
503	no_provider_available, not_ready	Health/circuit/dependency.
504	deadline_exceeded	End-to-end deadline.

Retry semantics

Retry only classified transient errors, only before visible output, and only within remaining attempts/cost/deadline. Use exponential backoff with full jitter and honor Retry-After. Transport reconnects before a billable acceptance may remain one attempt; any possibly billable re-invocation is a new attempt.

10. Routing policy and task matrix

Ordered eligibility pipeline

```
1 authentication/client policy
2 privacy/data-handling eligibility
3 required capabilities and endpoint support
4 context/output-window fit
5 risk quality floor and validator availability
6 allow/deny and override rules
7 health/circuit/quota
8 request/scoped budget feasibility
9 deadline feasibility
10 deterministic ranking
```

Initial score

```
score = Wc*expected_total_effective_cost
      + Wl*predicted_latency
      + Wq*quality_shortfall_risk
      + Wr*provider_failure_risk

effective_cost = monetary_cost + quota_scarcity_penalty
expected total includes probability-weighted repair/escalation and validation
```

Weights are versioned policy data. v0.1 uses conservative pass-rate priors with source/version; later empirical estimates can replace them without changing the API.

Initial task classes and acceptance gates

Class	Primary gate
classification	Allowed-label and confidence checks.
extraction	Schema, types, source spans/ranges.
transformation	Deterministic comparison/invariants.
summarization	Length, source coverage, unsupported-claim checks.
rewriting	Constraint/style checks; semantic preservation when required.
creative writing	Format constraints; usually no semantic judge.
factual QA	Grounding/citation verification when freshness matters.
general reasoning	Rubric; independent judge for high risk.
code generation	Compile/tests/static analysis.
code review/debugging	Reproduction/tests plus independent review by risk.
SQL generation/review	Parser, read/write safety, EXPLAIN/test fixture.
structured data	JSON Schema and business rules.
tool selection	Tool name/argument schema and permission policy.
grounded QA	Citation reachability and claim-source entailment.

11. Security and operations

Security/privacy

- Authenticate all non-health endpoints; hash stored keys; support rotation and scopes.
- Treat prompts, tool definitions, images, and outputs as confidential. Apply size limits early.
- Parameterized DB access and strict schemas. Never interpolate content into shell commands.
- CLI adapter uses stdin and argument arrays without a shell; isolated environment, bounded output, process-tree timeout.
- Redact authorization, provider keys, cookies, bodies, tool arguments, and common secrets from logs/errors/traces.
- Allowlist outbound destinations and require TLS verification.

Privacy	Eligible model class	Logging
public	public_only, standard, or zdr subject to policy	No raw content by default.
confidential	approved standard or zdr	No raw content; metadata allowlist.
deployment strict	zdr only	Minimal telemetry.

Metrics/tracing

Measure terminal state, route/model, attempts, validator result, estimated vs actual cost, reservation failures, latency by stage, provider errors, circuit state, quota staleness, and redaction failures. Avoid high-cardinality metadata. Trace request -> attempt -> validation by IDs, not content.

Readiness/recovery

- Readiness fails for pending migrations, no active policy, empty registry, or no required route.
- Expire orphan reservations only after reconciling attempt state.
- Never automatically re-invoke an ambiguous billable request without provider idempotency.

12. Testing strategy

Layer	Required coverage
Unit	Control precedence, normalization, Decimal cost, gates, ranking, state transitions, validation precedence, redaction.
Property-based	No budget overspend under concurrency; only legal states; serializer semantics; monotonic cost.
Contract	Golden fixtures for both APIs/SSE, SDK smoke tests, errors, tools, structured output.
Integration	SQLite/PostgreSQL, migrations, concurrent reservations, idempotency races, fake provider, cancellation.
Failure injection	429/5xx, malformed output, delayed chunks, partial disconnect, DB failure, stale quota, validator crash.
E2E	Pass, repair, escalate, explicit failure, no-route, budget denial, inspect parity, confidential exclusion, recovery.
Security	Scopes, size limits, SSRF allowlist, redaction snapshots, CLI injection, secret scan.
Performance	100 concurrent inspect; 20 orchestrations; lock duration; first-byte overhead; memory.

Essential scenarios

ID	Scenario	Expected
T01	Confidential request; cheapest model public_only	Excluded before reservation/invocation.
T02	Two requests compete for last budget	Exactly one reservation succeeds.
T03	Provider 429 before output	Bounded fallback and correct attempts.
T04	JSON fail, same-model repair passes	Two linked generations; one response.
T05	Stream fails after first token	FAILED_PARTIAL; no fallback.
T06	Same idempotency key/body	No second provider call; replay.
T07	Same key, different body	409 conflict.
T08	Inspect then execute on same snapshots	Top route/controls agree.
T09	Deadline during validation	Cancel/settle; 504.
T10	Explicit model, fallback false, validation fail	No escalation; exhausted.

13. Implementation milestones

Milestone	Deliverables	Exit gate
M0 skeleton	Package/config/CI, lint/type/test, app and health.	Local run; CI green; request IDs.
M1 persistence	Entities/repos, Alembic, DB matrix, seed registry/policies.	Migration and round-trip tests both DBs.
M2 API/canonical	Endpoints, models, strict schemas, auth/errors, normalize/serialize.	Golden contracts and SDK smoke.
M3 routing	Classifier/static fallback, policy, registry, scoring, inspect.	Eligibility/determinism suite.
M4 budgets/providers	Reservations, fake + one HTTP + CLI, retry/circuit.	Concurrency/failure suite.
M5 orchestration	State machine, validation, repair/escalation, idempotency/cancel.	Non-stream E2E suite.
M6 streaming	Both SSE protocols and first-token boundary.	Streaming failure/disconnect suite.
M7 hardening	Redaction, observability, reconciler, load/security, runbook.	Acceptance checklist.

Build order

Start with a deterministic fake provider and validators. Complete inspect and non-stream orchestration against fakes, then add one real provider end-to-end and subsequent adapters. This isolates state/budget correctness from provider variability.

Per-milestone definition of done

- Code, migration, configuration example, and tests land together.
- Public behavior has executable examples.
- New states/errors have redacted logs and metrics.
- No TODO weakens privacy, budget, idempotency, or state invariants.

14. v0.1 acceptance criteria

Functional

- Both generation endpoints work with model=auto for standard clients; /v1/models and /route/inspect behave as specified.
- Four configurable adapter paths: Gemini HTTP, CommandCode CLI, OpenAI-compatible HTTP, Anthropic HTTP, directly or via normalized transport.
- At least 10 task classes have policy fixtures, validation requirements, and escalation ladders.
- Structured JSON, tools passthrough, idempotency, retry/fallback, repair/escalation, cancellation, and streaming work.

Safety/correctness

- Tests prove privacy-ineligible providers are never invoked.
- Concurrent tests prove hard budgets cannot be oversubscribed in SQLite or PostgreSQL.
- No attempt begins when cost, count, deadline, privacy, capability, or quality would be violated.
- Every terminal path settles/releases reservations and persists terminal reason.
- Raw content and credentials are absent from default DB telemetry, logs, traces, and errors.

Quality/operations

- Successful requests expose request ID; authorized debug exposes resolved model, validation, attempts, latency, and Decimal actual cost.
- Every route freezes policy/version and snapshot time; tie-breaking is deterministic.
- Readiness, metrics, circuit breakers, reconciliation, and quota staleness are tested.
- Unit, contract, integration, failure, security, E2E tests run in CI; SQLite always, PostgreSQL on integration.

Release only when each criterion is demonstrated by an automated test or documented repeatable operational check. Live provider credentials/spend are not required for default CI.

15. Codex execution checklist

Step	Implementation task	Evidence
1	Inspect repo; preserve existing work; record assumptions.	No destructive changes.
2	Domain values and legal transition table.	Allowed/forbidden transition tests.
3	ORM, reservations/idempotency, repositories, migrations.	SQLite/Postgres tests.
4	Strict API models, canonical normalization, serializers, errors.	Golden Chat/Responses fixtures.
5	Registry, policies, scoring, inspect.	Determinism and hard-gate tests.
6	BudgetManager and concurrency.	No oversubscription/leaks.
7	Fake adapter/validators, orchestration loop.	Pass/repair/escalate/exhaust/cancel E2E.
8	Real adapters one at a time.	Fake-server contract tests; optional live smoke.
9	SSE and first-token boundary.	Disconnect/partial failure tests.
10	Auth, redaction, metrics, circuits, reconciler, docs.	Security/failure/load checks.
11	Acceptance matrix and release report.	CI artifacts and checklist.

Local implementation discretion

Codex may choose exact Python libraries, filenames, dependency-injection style, and test factories consistent with this architecture. Stop for direction before changing public semantics, weakening a hard invariant, storing raw content, or adding externally billed test actions.

Explicitly deferred

ML/contextual-bandit routing, distributed queues, Redis, UI, price scraping, multi-user invoicing, provider-hosted tools, background Responses, and commercial SaaS controls.

Final artifacts

- Source and migrations; seeded registry/policies; OpenAPI document; sample environment without secrets; architecture/state docs; CI suite; operational runbook; acceptance report.

End of v0.1 implementation contract